AI-Powered Smart Contract Bytecode Decompiler

Eyerusalem Hawoltu Afework
Computer Science, NYUAD
eh3115@nyu.edu

Advised by: Primary: Karim Ali, Secondary: Yasir Zaki

ABSTRACT

Less than one percent of deployed Ethereum smart contracts have publicly verified source code, leaving most on-chain logic opaque and difficult to audit. This project investigates whether modern large language models (LLMs) can help bridge the semantic gap between low-level Ethereum Virtual Machine (EVM) bytecode and high-level Solidity code.

To support this study, we assemble a large-scale dataset by cloning the Smart-Contract-Sanctuary-Ethereum repository (tintinweb), which contains hundreds of thousands of real-world mainnet contracts. From this corpus, we process 292,837 contract files, extract their deployed bytecode, reconstruct control-flow graphs (CFGs) using `evm_cfg_builder`, and pair each CFG with its corresponding Solidity function definition.

Using this aligned function-level dataset, we build a benchmarking pipeline evaluating three contemporary LLMs: GPT-5.1, Gemini 2.5 Flash, and DeepSeek-Reasoner, on the task of reconstructing Solidity-like functions directly from CFG-derived representations. Reconstruction quality is assessed using both structural and semantic metrics, including BLEU for syntactic similarity and CodeBERT cosine similarity for semantic alignment. While BLEU scores remain low across all models, semantic similarity is surprisingly high, indicating that LLMs often generate plausible high-level logic even when the output diverges from the exact ground truth.

Qualitative analysis shows that the models reliably reproduce simple ownership and configuration functions but consistently fail on more complex behaviors involving loops, dynamic arrays, inter-contract calls, or multi-step invariants.

This work provides one of the first systematic evaluations of LLM-based smart-contract decompilation using real mainnet data and highlights both the promise and the limitations of learned decompilation. The findings lay the groundwork for hybrid pipelines that combine static analysis with LLM reasoning, advancing the future of smart contract reverse engineering and security analysis.

KEYWORDS

Ethereum smart contracts, EVM bytecode, decompilation, control-flow graphs, large language models, CodeBERT, blockchain security, Solidity reconstruction, bytecode analysis

Reference Format:
Eyerusalem Hawoltu Afework. 2025. AI-Powered Smart Contract Bytecode Decompiler. In *NYUAD Capstone Seminar Reports, Spring 2025, Abu Dhabi, UAE.* 7 pages.

1 INTRODUCTION

Decentralized applications (dApps) and decentralized finance (DeFi) rely heavily on smart contracts, which collectively secure and move billions of dollars across public blockchains. Despite this scale, more than 99% of deployed Ethereum contracts lack publicly verified source code [3]. Once compiled into Ethereum Virtual Machine (EVM) bytecode, a contract loses its high-level structure, type information, and developer intent, leaving auditors, developers, and users with only low-level opcodes to reason about. This opacity severely limits transparency and makes security analysis, incident response, and forensic investigations significantly more difficult.

Traditional attempts to reverse-engineer smart contracts rely on static-analysis–based decompilation tools such as Dedaub [7] and Panoramix [10]. Although these systems recover partial control-flow structure, they often produce incomplete, noisy, or misleading output—particularly for modern optimized bytecode or contracts involving complex financial logic. Prior work suggests that static approaches alone are fundamentally limited in their ability to reconstruct high-level semantics from EVM execution patterns [2]. These limitations motivate exploring learning-based approaches that can generalize beyond hand-crafted heuristics.

This report is submitted to NYUAD's capstone repository in fulfillment of NYUAD's Computer Science major graduation requirements.

جامعة نيويورك أبوظبي
NYU ABU DHABI

Capstone Seminar, Spring 2025, Abu Dhabi, UAE

Recent advances in large language models (LLMs) have demonstrated strong capabilities in code understanding, translation, and synthesis, exemplified by GPT-3 [1], CodeT5+ [12], and more recent reasoning-enhanced models. Inspired by recent work framing large language models as general pattern-translation systems [6], this project investigates whether modern LLMs can perform *decompilation by reconstruction*: generating plausible Solidity-like code directly from low-level bytecode structure.

To enable this study, we construct a large-scale dataset by cloning the Smart-Contract-Sanctuary-Ethereum repository [9], which archives hundreds of thousands of real mainnet contracts. From this corpus, we curate a filtered subset of 292,837 contract files by removing legacy compiler versions and eliminating contracts with unresolved dependencies. For each remaining contract, we extract all available Solidity function definitions and pair each function with a control-flow graph (CFG) reconstructed from its compiled bytecode using `evm_cfg_builder` [8]. This produces an aligned dataset in which each function is represented by both its bytecode-derived CFG and its high-level Solidity implementation.

Using this dataset, we evaluate three contemporary LLMs: GPT-5.1, Gemini 2.5 Flash, and DeepSeek-Reasoner on the task of reconstructing Solidity functions solely from CFG representations. This approach departs from earlier sequence-to-sequence decompiler designs by testing whether state-of-the-art foundation models can perform semantic recovery *zero-shot*, without any fine-tuning.

Compilation is inherently lossy: variable names, comments, type annotations, and higher-level control structures are removed during the source-to-bytecode transformation. Recovering these abstractions requires models capable of understanding opcode patterns, long-range dependencies, and control-flow semantics. By examining LLM performance on this structured dataset, this project explores the extent to which machine learning can bridge the semantic gap between EVM bytecode and human-readable logic, pointing toward future hybrid systems that combine static analysis with LLM-based reasoning for more effective smart contract security.

2 RELATED WORK

Several research efforts have explored smart contract analysis, decompilation, and the use of large language models (LLMs) for code understanding. This section highlights related work across three main domains: smart contract decompilers, vulnerability detection, and Transformer-based machine learning models for code generation.

Dedaub [7] is a leading smart contract decompiler that reconstructs high-level contract logic from EVM bytecode using static analysis and control flow graph recovery. It produces human-readable pseudo-code and has been widely used in both research and industry. However, its rule-based design limits its adaptability to previously unseen or obfuscated bytecode structures. Similarly, Panoramix [10] offers a Python-based decompiler and disassembler. It disassembles contracts into a pseudo-assembly form and reconstructs function boundaries using symbolic execution and jump resolution. While effective in certain cases, its output lacks Solidity-like readability and fails on many edge cases or newer compiler patterns.

More recently, researchers have explored using LLMs to analyze or translate smart contracts. Darwish [2] conducted a study using GPT-3.5, GPT-4, and CodeLlama to evaluate vulnerabilities in smart contracts decompiled with Dedaub [7]. This thesis demonstrated the potential of language models to analyze security flaws but did not attempt to train models on raw bytecode for reverse compilation. Our project builds on this idea by evaluating LLMs directly on aligned bytecode–source code pairs, eliminating the dependency on external decompilers.

The foundational work on LLMs comes from Brown et al. [1], who introduced GPT-3, a transformer-based model that demonstrated strong few-shot learning and code generation capabilities. This inspired a wave of models specialized for code tasks. Among these, CodeT5+ [12] is one of the most notable open-source models trained specifically for program understanding, synthesis, and translation. We reference CodeT5+ as part of the broader landscape of code-focused language models, though our evaluation centers on general-purpose reasoning models rather than task-specific code LLMs.

In addition to academic work, Etherscan [3] plays a central role in the Ethereum ecosystem as a public block explorer and verification service, where contract developers can publish source code corresponding to on-chain bytecode. Our dataset instead builds on the Smart-Contract-Sanctuary-Ethereum repository [9], which archives mainnet contracts and their verified sources at scale, allowing us to construct aligned bytecode–source code pairs for our study.

Overall, while tools like Dedaub and Panoramix provide strong static analysis foundations, they lack adaptability. For instance, Dedaub may fail to correctly decompile contracts that use non-standard control flow or newer Solidity compiler features such as Yul-based inline assembly, producing incomplete or misleading pseudo-code. Similarly, Panoramix struggles with contracts that contain heavily obfuscated bytecode or customized proxy logic, leading to incorrect function boundary reconstruction. Prior LLM-based studies focused primarily on analysis rather than decompilation. Our work differs in that we evaluate modern LLMs directly on aligned bytecode–source code pairs, investigating whether they can

reconstruct high-level semantics without relying on predefined decompilation rules.

3 METHODOLOGY

This section describes the full pipeline used to construct the dataset, extract control-flow graphs (CFGs) from EVM bytecode, generate Solidity reconstructions using large language models (LLMs), and evaluate their performance using both syntactic and semantic metrics. The methodology consists of four stages: dataset construction, CFG extraction, model prompting, and evaluation.

3.1 Overview

The overall workflow proceeds as follows:

(1) Collect verified Solidity contracts from a large public repository [9].
(2) Group contracts by compiler version and compile them to bytecode.
(3) Extract function-level CFGs using static analysis tools such as `evm_cfg_builder` [8].
(4) Align each CFG with its corresponding Solidity source function.
(5) Query multiple LLMs to reconstruct Solidity functions from CFGs.
(6) Evaluate reconstructed functions using BLEU [11] and CodeBERT similarity [4].

This setup simulates a realistic reverse-engineering task in which the model sees only CFG information derived from runtime bytecode and must reconstruct a high-level Solidity implementation.

3.2 Dataset Construction

3.2.1 Collecting Contracts and Grouping by Version. The dataset construction begins with a large-scale clone of the Smart Contract Sanctuary Ethereum repository [9], which hosts hundreds of thousands of real-world mainnet contracts.

```
contracts/mainnet/
  000/
  001/
  ...
```

From this corpus, all Solidity source files (`*.sol`) are extracted, cleaned, and grouped by compiler version based on their `pragma solidity` declarations. Each compiler version is placed into its own directory:

```
Contracts_By_Version/
  0.8.0/
  0.8.4/
  0.8.7/
  0.8.9/
  ...
```

After filtering out unsupported legacy versions and removing contracts with unresolved dependencies, the final dataset contains **292,837** Solidity contract files spanning multiple minor versions of the compiler.

3.2.2 Compiling Contracts. To reproduce the deployed bytecode as faithfully as possible, each contract is compiled using the exact Solidity compiler version extracted from its pragma. This is performed using `solc-select`, falling back to the closest compatible patch version when needed.

Compilation yields:

- runtime bytecode files (`*.hex`),
- compilation logs, and
- failure reports.

This ensures consistency between the original on-chain bytecode and the bytecode used for CFG construction.

3.2.3 Extracting Control-Flow Graphs. CFGs are extracted using `evm_cfg_builder` [8], a static analysis tool provided by Trail of Bits. Because the upstream tool has not been actively maintained in recent years, its built-in function-selector mapping is incomplete. To address this, we extend the pipeline by integrating a curated list of 55,674 known selector–signature pairs (e.g., `0x70a08231 → balanceOf(address)`, `0xa9059cbb → transfer(address,uint256)`). These additional entries allow the dispatcher reconstruction stage to correctly resolve many selectors that `evm_cfg_builder` would otherwise treat as unknown, improving the accuracy of function-boundary identification.

For each bytecode file, the tool outputs:

- a dispatcher graph (mapping selectors to code blocks),
- a full CFG (`.dot` representation),
- a serialized textual form listing basic blocks and opcodes.

Each CFG captures fundamental control-flow information such as `JUMP`, `JUMPI`, function entry points, revert blocks, and calldata loaders. These structural elements form the input to all LLMs.

3.3 Function-Level Alignment

For each contract, all function definitions are parsed from the Solidity source. Each function is matched to its CFG representation based on function selectors and dispatch structure. Each aligned entry takes the following JSON form:

```
{
  "withdrawForeignERC721(address,uint256)": {
    "solidity_definition":
         "function withdrawForeignERC721(...) {...}",
    "cfg_representation":
        "Function withdrawForeignERC721.\nBasic Blocks: ..."
  }
}
```

A normalized JSONL dataset is also created, where each line is a single function instance:

```
{
  "version": "0.8.9",
  "function_name":
        "toggleSwap(bool)_v175",
  "cfg_representation": "...",
  "solidity_definition":
        "function toggleSwap(bool _swapEnabled) ..."
}
```

The suffix (e.g., _v175) uniquely distinguishes functions with identical signatures across different contracts.

3.4 Decompilation Task Definition

The core research task is:

> *Given only a textual CFG generated from EVM bytecode, reconstruct the most likely Solidity implementation of the function.*

The reconstruction must:

- preserve the original function signature,
- infer visibility, modifiers, and revert conditions,
- reflect the control-flow implied by the CFG,
- resemble the ground-truth Solidity implementation.

This framing mirrors a real reverse-engineering scenario.

3.5 Model Selection and Prompting

Three contemporary LLMs are evaluated:

- **GPT-5.1**: a frontier model with strong code reasoning capabilities.
- **Gemini 2.5 Flash**: a high-speed model optimized for efficiency.
- **DeepSeek-Reasoner**: a reasoning-oriented model especially strong on structured tasks.

All models receive the same structured prompt. An example prompt is shown below:

```
You will receive an Ethereum EVM opcode
Your task is to reconstruct the
**full Solidity function**, including:

- The function name
- All parameters and types
- Visibility (public, external, internal, private)
- Modifiers (onlyOwner, payable, view, etc.)
- The full function body (based on best inference)
- Emit statements, requires, assignments, returns, etc.

Your output
**must be a complete valid Solidity function**.

Use the few-shot examples to learn
how CFG patterns map to real functions.

[Rules]
1. Output only the Solidity function. No explanation.
2. Your answer must be a complete function enclosed in:
   function name(...) visibility ... {
       ...
   }
3. Infer as much as possible from CALLDATALOAD, SLOAD,
CALLER, EQ, conditions, etc.
4. If uncertain about exact variable names, use safe,
generic names like: param1, param2, account,amount,
newOwner, etc.
5. Do NOT truncate the function body. Always output a
complete function.

[Few-shot Examples]

Example 3
CFG:
Function <UNKNOWN>
Attributes:
-payable
-view

Basic Blocks:
- @0x29b-0x2a2
Instructions:
- JUMPDEST
- PUSH2
- PUSH2
- JUMP
Incoming basic_block:
Outgoing basic_block:
- <cfg BasicBlock@0x807-0x80e>

- @0x807-0x80e
Instructions:
- JUMPDEST
- PUSH2
- PUSH2
....
- JUMP
Incoming basic_block:
- <cfg BasicBlock@0xd9b-0xdad>
Outgoing basic_block:
- <cfg BasicBlock@0xb77-0xb7f>

Solidity:
function renounceOwnership() public virtual onlyOwner {
        _transferOwnership(address(0));
}
```

```
Reconstruct the most likely original Solidity
implementation. Return only the Solidity
function.
```

Temperature is set to 0 for determinism. No chain-of-thought or examples are included.

4 EVALUATION

This section describes the methodology used to evaluate the ability of large language models (LLMs) to reconstruct Solidity functions from control-flow–graph (CFG) representations derived from EVM bytecode. The evaluation consists of four components: the overall strategy, the experimental setup, the hypotheses and their outcomes, and visualizations of the quantitative results.

4.1 Evaluation Strategy

The central question of this study is whether an LLM can produce a meaningful Solidity reconstruction when given only a CFG extracted from EVM bytecode. To evaluate this, I adopt a dual-metric framework that combines syntactic similarity and semantic similarity. To ensure a fair comparison across all models, a unified CSV dataset is constructed containing the CFG, the ground-truth Solidity function, and the corresponding outputs from GPT-5.1, Gemini 2.5 Flash, and DeepSeek-Reasoner:

```
cfg, actual, gpt_output, gemini_output,
deepseek_output
```

A second lightweight dataset omits the CFG for simplified manual comparison:

```
actual_function, gpt_output, gemini_output,
deepseek_output
```

This two-part structure supports both quantitative scoring and qualitative inspection.

4.2 Evaluation Metrics

4.2.1 BLEU: Syntactic Similarity. BLEU [11] measures surface-form similarity via n-gram overlap between generated and reference code. Although originally designed for machine translation, it is frequently used for code generation tasks. However, BLEU is sensitive to:

- renaming of variables,
- formatting or whitespace changes,
- benign paraphrasing,
- reordered but semantically equivalent statements.

Because of this strictness, BLEU often underestimates the quality of logically correct but syntactically different completions. This behavior is reflected in our results: all models achieve relatively low BLEU-4 scores.

The average BLEU-4 scores across the evaluated dataset are:

Model	BLEU-4 Score
GPT-5.1	11.91
Gemini 2.5 Flash	8.80
DeepSeek-Reasoner	19.64

DeepSeek-Reasoner performs best syntactically, though overall BLEU values remain low, confirming that LLM-generated Solidity rarely matches the ground truth token-by-token.

4.2.2 CodeBERT: Semantic Similarity. To evaluate deeper structural correctness, CodeBERT [4] is used as an embedding model. Semantic similarity is computed via cosine similarity:

$$\text{sim}(g, \hat{g}) = \frac{E(g) \cdot E(\hat{g})}{\|E(g)\| \, \|E(\hat{g})\|}$$

Where $E(\cdot)$ denotes the CodeBERT encoder applied to a code snippet.

Average similarity scores across models are:

Model	Average CodeBERT Similarity
GPT-5.1	0.9781
DeepSeek-Reasoner	0.9704
Gemini 2.5 Flash	0.9590

These values demonstrate high semantic alignment even when BLEU is low. This suggests that LLMs often recover the correct logical pattern or intent despite syntactic deviations.

Why BLEU + CodeBERT?.

- **Exact string matching or AST equivalence** is too strict because compilation is lossy.
- **Symbolic equivalence or formal methods** require full contract context and executable Solidity, which is unavailable for many entries.
- **Human evaluation** is costly and difficult to reproduce.

BLEU + CodeBERT therefore provides a scalable, reproducible, and meaningful evaluation for decompilation.

4.3 Experimental Setup

The experimental process consists of:

(1) **Dataset.** A set of 25 aligned CFG–Solidity function pairs is sampled from the 292,837 processed contracts, spanning Solidity versions 0.8.0–0.8.24 and covering patterns such as ownership modifiers, fee configuration, allowance management, and liquidity operations.
(2) **Models.** Three state-of-the-art LLMs are tested:
   - GPT-5.1 (OpenAI)
   - Gemini 2.5 Flash (Google DeepMind)
   - DeepSeek-Reasoner (DeepSeek)

Models are evaluated *zero-shot*, with no fine-tuning.

(3) **Prompting.** Each model receives:
   (a) the Solidity compiler version,
   (b) the textual CFG representation produced by `evm_cfg_builder`,
   (c) and instructions to output only the reconstructed Solidity code.

   Although the dataset contains the ground-truth function signatures taken from verified Solidity source, these signatures are *purposefully withheld* during evaluation. As a result, the CFG input appears with `Function <UNKNOWN>` and contains no ABI metadata. This intentional masking forces the LLMs to infer the function name, parameter types, visibility, and return values directly from bytecode-level control-flow— mirroring a realistic decompilation scenario where ABI information is not available.
(4) **Scoring.** BLEU-1 through BLEU-4 are computed using NLTK; CodeBERT embeddings provide semantic similarity.
(5) **Aggregation.** Scores are averaged across all functions for cross-model comparison.

Together, these results illustrate a clear pattern: LLMs recover the *intent* of Solidity functions far better than their exact syntax. This highlights the promise of LLM-assisted decompilation while emphasizing the need for hybrid approaches that combine static analysis with semantic reasoning for high-stakes security analysis.

5 PROJECT TIMELINE

(1) **Literature Review and Problem Definition (Week 1–2):** This phase involved surveying prior work on EVM bytecode analysis, decompilation, and smart contract security, including tools such as Dedaub [7] and Panoramix [10], as well as LLM-based program translation methods such as GPT-3 [1], CodeT5+[12], and the assembly-to-Solidity model Nova[5]. This review clarified the research gap and established the central question of whether modern LLMs can reconstruct high-level Solidity logic from CFG-derived EVM bytecode.
(2) **Dataset Acquisition from Public Repositories (Week 3):** The Smart Contract Sanctuary repository was cloned, providing a large archive of verified contracts. A total of 292,837 Solidity files were collected across many compiler versions, and metadata such as pragma directives was extracted to support version-based grouping.
(3) **Bytecode Compilation and CFG Extraction (Week 4–6):** All collected contracts were compiled using `solc-select` with fallback version resolution. Runtime bytecode was produced, and control-flow graphs (CFGs) were extracted using a modified version of `evm-cfg-builder`. Compilation failures, malformed files, and timeouts were logged using a resumable pipeline to ensure full dataset coverage.
(4) **Function-Level Alignment and JSON Construction (Week 7–8):** Each CFG was aligned with its corresponding Solidity function definition. Structured JSON and JSONL entries were generated containing the Solidity version, function name, CFG representation, and ground-truth source code. This formed the aligned dataset used for model benchmarking.
(5) **Prompt Engineering and LLM Benchmark Setup (Week 9–10):** A standardized prompt template for CFG-based function reconstruction was developed. Four models: GPT-5.1, Gemini 2.5 Flash, DeepSeek-Reasoner, and Nova [5] were prepared for evaluation. Model outputs were saved in synchronized JSON and JSONL files to guarantee consistent input conditions.
(6) **Evaluation and Scoring (Week 11–12):** Reconstructed functions were evaluated using both surface-level and semantic metrics. BLEU [11] scores (BLEU-1 through BLEU-4) measured token-level similarity, while CodeBERT embeddings [4] quantified semantic similarity. This produced quantitative results for GPT-5.1, Gemini 2.5 Flash, and DeepSeek-Reasoner, with qualitative comparisons including Nova.
(7) **Final Report, Editing, and Presentation Preparation (Week 13-14):** Final polishing and documentation tasks were completed: refining the LaTeX report, verifying citations, preparing presentation slides, and producing the final PDF for committee submission.

6 BUDGET

To successfully complete this capstone project, several external services were required to support large-scale data processing, blockchain infrastructure access, and model evaluation, all of which incur direct monetary costs. I purchased DeepSeek API tokens on November 14, 2025 for $2.12 (including VAT) to run CFG-to-Solidity decompilation experiments using the DeepSeek-Reasoner model, and OpenAI API usage credits on November 18, 2025 for $10.50 (including 5% VAT) to evaluate GPT-5.1 on the same benchmark. These credits were essential for generating model outputs across hundreds of CFG samples and for computing semantic evaluation metrics. In addition, I maintained an active ChatGPT Plus monthly subscription, renewed through Apple for AED 79.99 (approximately $21.77).

7 CONCLUSION

This project investigates whether modern large language models can serve as semantic decompilers for Ethereum

smart contracts by reconstructing high-level Solidity code from low-level control-flow representations extracted from EVM bytecode. To enable this study, a large function-level dataset was constructed consisting of 292,837 real-world contracts collected from the Smart Contract Sanctuary repository. These contracts were grouped by Solidity version, compiled, and analyzed using `evm-cfg-builder` to obtain textual CFG representations. Each CFG was aligned with its corresponding ground-truth Solidity function, producing a rich benchmark for evaluating LLM-based decompilation.

Using this dataset, three contemporary LLMs: GPT-5.1, Gemini 2.5 Flash, and DeepSeek-Reasoner were evaluated on the task of reconstructing Solidity functions purely from CFG text in a zero-shot setting. Quantitative evaluation shows that surface-level token overlap is low across all models (BLEU-4 in the 10–20 range), confirming that none of the models reproduce the original source code verbatim. However, semantic similarity measured using CodeBERT embeddings [4] is remarkably high (0.958–0.978), indicating that even when syntactically different, the generated code frequently captures the intended behavior or structural pattern of the original implementation.

Qualitative analysis reinforces these findings. All three models reliably reconstruct simple ownership functions, flag toggles, and common ERC-20 patterns such as allowance management. At the same time, they struggle with complex multi-call functions, loops over dynamic arrays, intricate liquidity routines, and highly optimized DeFi logic. In these scenarios, the models tend to hallucinate helper functions, simplify multi-step logic, or omit important invariants, demonstrating the limits of CFG-only inputs and zero-shot reconstruction for faithfully restoring high-level semantics.

Overall, this work provides three core contributions:

(1) a scalable pipeline for aligning real-world Solidity functions with CFGs extracted from bytecode,
(2) a multi-model benchmark for evaluating LLM-based decompilation performance, and
(3) a systematic empirical analysis of the strengths and weaknesses of contemporary LLMs in this setting.

The results demonstrate that large language models can meaningfully assist in understanding smart contract behavior especially for common structural patterns but they cannot yet replace traditional static analysis tools or guarantee faithful reconstructions in security-critical contexts.

Future progress will likely require hybrid techniques that integrate static analysis (e.g., data-flow reasoning, symbolic stack inspection, type inference) with LLM-based semantic modeling, as well as fine-tuned models trained directly on large-scale bytecode–source pairs. Nevertheless, this project establishes a foundational benchmark and demonstrates that LLM-based decompilation is both feasible and promising, offering a new pathway toward greater transparency, accessibility, and security in the smart contract ecosystem.

REFERENCES

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.14165.

[2] M. Darwish. From bytecode to safety: Decompiling smart contracts for vulnerability analysis. https://www.diva-portal.org/smash/get/diva2:1864948/FULLTEXT01.pdf, 2023. Bachelor's Thesis, DiVA Portal.

[3] Etherscan. Etherscan API v2 Documentation. https://docs.etherscan.io/etherscan-v2, 2025. Accessed: 2025-05-07.

[4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL https://arxiv.org/abs/2002.08155.

[5] Nan Jiang, Chengxiao Wang, Kevin Liu, Xiangzhe Xu, Lin Tan, Xiangyu Zhang, and Petr Babkin. Nova: Generative language models for assembly code with hierarchical attention and contrastive learning. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://www.cs.purdue.edu/homes/lintan/publications/nova-iclr25.pdf.

[6] Suvir Mirchandani, Fei Xia, Pete Florence, Brian Ichter, Danny Driess, Montserrat Gonzalez Arenas, Kanishka Rao, Dorsa Sadigh, and Andy Zeng. Large language models as general pattern machines. *arXiv preprint arXiv:2307.04721*, 2023. URL https://arxiv.org/abs/2307.04721.

[7] G. Neville et al. Dedaub: High-level decompilation of smart contracts. In *IEEE Symposium on Security and Privacy*, 2023. URL https://ieeexplore.ieee.org/document/10123564.

[8] Trail of Bits. evm_cfg_builder. https://github.com/crytic/evm_cfg_builder, n.d. GitHub repository.

[9] Matthias Ortner and Shayan Eskandari. Smart contract sanctuary. https://github.com/tintinweb/smart-contract-sanctuary, n.d. GitHub repository.

[10] Palkeo. Panoramix. https://github.com/palkeo/panoramix, 2023. GitHub Repository.

[11] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318. ACL, 2002. URL https://aclanthology.org/P02-1040.

[12] Yao Wang et al. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2305.07922.